

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE (SRS)

Proyecto: Plataforma Integral de Gestión y Reporteo Trimestral (REDECO/CONDUSEF)

Institución: Unión de Crédito Integral, S.A. de C.V. (UCISA)

Curso/Asignatura: Negocios Electrónicos.

Fecha de Publicación: 13 de junio del 2026

Versión: 1.0.0

CONTROL DE VERSIONES Y AUTORES

Autores / Integrantes del Proyecto:

1. Adán Gurrola Grijalva - 38198801
2. Eduardo Octavio Picazo Méndez - 42208075
3. Fredy Rodriguez Rodarte - 42203151

Historial de Revisiones:

Versión	Fecha	Descripción del Cambio	Autor(es)	Revisor / Aprobador
1.0.0	13-jun.-26	Creación del documento base SRS derivado del análisis estático de código fuente y base de datos relacional.	Adán Gurrola. Eduardo Octavio. Fredy Rodriguez	Carlos Eduardo Acuña López

1. INTRODUCCIÓN

1.1 Propósito del Documento

El presente documento tiene como objetivo formalizar y detallar la Especificación de Requisitos de Software (SRS, por sus siglas en inglés, *Software Requirements Specification*) para el desarrollo e implementación de la Plataforma Integral de Gestión y Reporteo Trimestral orientada a cumplir con la regulación REDECO de la Comisión Nacional para la Protección y Defensa de los Usuarios de Servicios Financieros (CONDUSEF).

Esta especificación se redacta bajo la adaptación académica del estándar IEEE 830, definiendo con precisión científica y técnica el comportamiento funcional, los límites operacionales, las restricciones del sistema y los atributos de calidad no funcionales del aplicativo. El documento está dirigido al equipo de desarrollo de software, al personal de auditoría e ingeniería de requisitos, a los evaluadores académicos y a la gerencia de TI de la Unión de Crédito Integral, S.A. de C.V. (UCISA).

1.2 Alcance del Sistema

La plataforma desarrollada (denominada en lo sucesivo "el Sistema" o "REDECO-UCISA") es una solución web de tipo Monolito Modular diseñada para automatizar la captura, control, validación geográfica y empaquetamiento de reclamaciones, quejas e incidencias financieras reportadas por los clientes o socios de la institución.

1.2.1 Capacidades Incluidas (Lo que el software HACE):

- 1. Administración y Registro de Socios:** Mantenimiento (Creación, Lectura, Actualización y Eliminación - CRUD) de la base de datos de socios o entidades afiliadas a la Unión de Crédito, controlando variables obligatorias de identificación regulatoria como la Clave CONDUSEF.
- 2. Geolocalización Predictiva Integrada:** Mecanismo de consulta asíncrona sobre la base de datos nacional SEPOMEX (Servicio Postal Mexicano) que permite, mediante el ingreso de un Código Postal de 5 dígitos, autocompletar la estructura geográfica de los socios (Estado de la República, Delegación o Municipio, y Colonias asociadas).
- 3. Captura Dinámica de Quejas:** Interfaz gráfica para la recolección de quejas asociadas a productos y causas. Incluye validaciones cruzadas en tiempo de ejecución (tanto en el cliente mediante JavaScript nativo, como en el servidor a través de la capa de control de formularios de Django).
- 4. Validación de Fechas y Cierres:** Control estricto de integridad de datos que exige una Fecha de Cierre válida obligatoriamente cuando el estado de la reclamación se actualice a "Concluido/Cerrado".
- 5. Generación Automática de Folio Regulator:** Algoritmo determinista que genera folios secuenciales no colisionantes con el formato YYMMNN (Año de 2 dígitos + Mes de 2 dígitos + Consecutivo numérico mensual auto-incremental de 2 dígitos) a partir de la fecha de consulta de la queja.
- 6. Emisión de Acuses de Recibo (Tickets):** Generación de vouchers oficiales de quejas con hojas de estilos adaptadas para impresión física o guardado en formato digital (PDF) optimizando la visualización mediante la exclusión de elementos de interfaz de usuario de navegación web.
- 7. Cierre Trimestral y Empaquetado de Lotes:** Panel de auditoría temporal que permite realizar el filtrado cronológico de reclamaciones por Año y Trimestre fiscal (T1, T2, T3, T4), con capacidad de selección múltiple (checkboxes) para exportar de manera atómica un lote en formato JSON compatible con los esquemas de transmisión oficiales de la CONDUSEF.

1.2.2 Capacidades Excluidas (Lo que el software NO HACE):

1. **Transmisión Directa por API REST a CONDUSEF:** El sistema no realiza la petición HTTP POST en tiempo real hacia el *endpoint* del servidor de la CONDUSEF. En su lugar, el flujo concluye con la generación del archivo JSON estructurado para su carga manual o posterior integración de servicios batch de envío.
2. **Autenticación Multi-Factor (MFA) o Proveedor de Identidad Externo:** El sistema delega la seguridad de acceso a la suite interna de usuarios de Django, sin integraciones SAML 2.0 u OIDC.
3. **Gestión de Archivos Adjuntos / Evidencia Multimedia:** No se incluye la capacidad de subir documentos PDF, JPG o audios de llamadas en el registro de quejas.

1.3 Definiciones, Acrónimos y Abreviaturas

A continuación, se listan los términos fundamentales identificados a nivel de código y base de datos:

- **CONDUSEF:** Comisión Nacional para la Protección y Defensa de los Usuarios de Servicios Financieros. Organismo público descentralizado del gobierno mexicano que regula a las instituciones financieras.
- **REDECO:** Registro de Despachos de Cobranza. Plataforma administrada por la CONDUSEF para registrar las quejas presentadas por los usuarios relativas a la gestión de cobro de los créditos.
- **UCISA:** Unión de Crédito Integral, S.A. de C.V. Institución de crédito de segundo piso y objeto del presente desarrollo de software.
- **Socio / Institución:** Entidades, sucursales o filiales que forman parte de la Unión de Crédito y que pueden recibir reclamaciones financieras.
- **Folio ('YYMMNN'):** Identificador alfanumérico único para el registro regulatoria de la reclamación.
- **SEPOMEX:** Servicio Postal Mexicano. Entidad proveedora del catálogo geográfico oficial de códigos postales, municipios y colonias en México.
- **ORM (Object-Relational Mapping):** Mapeo Objeto-Relacional. Técnica de programación para convertir datos entre el sistema de tipo de lenguaje (Python) y la base de datos relacional (SQLite).
- **SSR (Server-Side Rendering):** Renderizado en el Servidor. Generación del documento HTML dinámico directamente en el backend (Django templates) antes de ser enviado al cliente.
- **CSRF (Cross-Site Request Forgery):** Vulnerabilidad y mecanismo de defensa que asegura que las solicitudes POST provienen de usuarios autenticados dentro del propio sistema mediante el uso de tokens criptográficos únicos por formulario.
- **JSON (JavaScript Object Notation):** Formato ligero de intercambio de datos, empleado para empaquetar los lotes de quejas y descargarlos como archivos planos.

1.4 Referencias

1. Comisión Nacional para la Protección y Defensa de los Usuarios de Servicios Financieros (CONDUSEF). *Especificación técnica para la interconexión con el sistema REDECO (API REDECO)*.
 2. Institute of Electrical and Electronics Engineers (IEEE). *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*.
 3. Django Project Documentation (v6.0.x). *Official Guide for Models, Forms and Views Workflow*. Recuperado de: <https://docs.djangoproject.com/>
 4. Código Postal Mexicano (SEPOMEX). *Catálogo de Códigos Postales e Información Geográfica*.
-

2. DESCRIPCIÓN GENERAL

2.1 Perspectiva del Producto

El aplicativo REDECO-UCISA se concibe como un software de gestión y consolidación regulatoria interno. Es un sistema predominantemente independiente (Stand-alone) en su capa de almacenamiento y procesamiento local, pero interactúa de forma asíncrona ("Loose Coupling") con los sistemas de la CONDUSEF. El sistema actúa como un middleware de datos financieros: extrae información de catálogos oficiales y de socios, procesa las transacciones en una base de datos local y genera el formato JSON para la comunicación con la API externa del regulador.

La base de datos SQLite embebida (`db.sqlite3`) almacena los catálogos estáticos de productos financieros (tales como Crédito AVIO, Refaccionario, con Aval, etc.), las causas regulatorias tipificadas por CONDUSEF, la tabla de socios activos y la base histórica de códigos postales nacionales provenientes de SEPOMEX.

2.2 Funciones del Producto

El sistema ejecuta cinco macro-funciones esenciales, deducidas del flujo lógico de controladores y plantillas:

1. **Módulo de Mantenimiento Geográfico (SEPOMEX):** Permite indexar, consultar y mapear los códigos postales del país para asegurar que la colonia y municipio del socio correspondan exactamente a los catálogos oficiales.
2. **Módulo CRUD de Socios Financieros:** Permite crear, listar, modificar y archivar lógicamente a los socios asociados a UCISA, capturando su clave regulatoria única.
3. **Módulo de Registro de Reclamaciones:** Captura de manera digital los datos de la queja (fecha de consulta, medio de recepción, producto financiero, causa tipificada y socio responsable).

4. **Módulo de Emisión de Acuses:** Genera vistas físicas preparadas para impresión con exclusión de elementos CSS multimedia de pantalla, sirviendo de comprobante oficial de registro.

5. **Módulo de Auditoría y Cierre Trimestral:** Filtra cronológicamente la información histórica de quejas y compila múltiples quejas validadas en un único documento de salida estructurado en JSON.

2.3 Características de los Usuarios

Basado en los perfiles requeridos para la operación del sistema y la configuración del panel administrativo de Django, se identifican dos perfiles formales de usuario:

1. Operador / Capturista de Quejas (Usuario Estándar):

- **Perfil Técnico:** Conocimientos básicos en herramientas web de escritorio.
- **Funciones en el Sistema:** Registro diario de incidencias financieras, consulta geográfica predictiva por código postal, impresión de acuses oficiales de quejas y filtrado de periodos trimestrales.
- **Nivel de Acceso:** Limitado a la interfaz de usuario del portal UCISA.

2. Administrador de Sistemas / Oficial de Cumplimiento Regulatorio:

- **Perfil Técnico:** Experiencia avanzada en gestión de plataformas TI y bases de datos relacionales.
- **Funciones en el Sistema:** Carga y depuración masiva de los catálogos de productos y causas, alta y modificación de socios en el panel administrativo de Django (`/admin/`), y generación del archivo JSON para su envío a CONDUSEF.
- **Nivel de Acceso:** Control total de lectura, escritura y eliminación de registros históricos en la base de datos relacional.

2.4 Restricciones Generales

1. **Persistencia Monolítica Exclusiva:** El almacenamiento se limita a un único nodo de base de datos relacional SQLite (`db.sqlite3`). No se contempla escalabilidad horizontal mediante réplicas de lectura de base de datos en caliente.

2. **Sincronismo de Guardado:** El guardado y generación de folios se realizan de forma atómica en el hilo principal del servidor de aplicaciones de Django, lo que limita la concurrencia masiva en ráfagas de inserción.

3. **Dependencia del Navegador del Cliente:** La lógica de autocompletado y renderizado dinámico de la fecha de cierre depende de la ejecución de JavaScript en el lado del cliente (Vanilla JS). El bloqueo de scripts en el navegador invalida la experiencia de usuario y traslada la validación exclusivamente a la capa del servidor.

4. **Dependencia de Datos Estáticos para Catálogos:** Los códigos de causas (por ejemplo, el código regulatoria *1211* "Consulta de estado de cuenta") y productos deben estar previamente insertados vía seeders para que el formulario de quejas sea operable.

3. REQUISITOS ESPECÍFICOS

3.1 Requisitos Funcionales

3.1.1 Módulo 1: Gestión de Socios e Instituciones

- **RF-01: Alta de Socios Financieros**

- **Descripción:** El sistema debe permitir registrar nuevos socios financieros recopilando información de identificación y localización geográfica.
- **Entradas:** Clave CONDUSEF (Cadena de texto, única), Nombre de la Institución (Cadena de texto), Tipo de Entidad (Elección obligatoria: 'Caja de Ahorro', 'Sucursal', 'Sofipo', 'Banco', 'Otro'), Código Postal (Cadena de 5 dígitos), Estado de la República (Elección obligatoria de catálogo de 32 estados), Municipio (Cadena de texto), Colonia (Cadena de texto) y Estatus ('Activo' o 'Inactivo').
- **Proceso:** Validar que los campos obligatorios no estén vacíos y que la Clave CONDUSEF no esté duplicada. Si la validación es correcta, guardar el registro en la tabla `socio`.
- **Salida:** Confirmación de inserción en la interfaz de usuario mediante un mensaje de éxito ("Socio agregado correctamente") y recarga de la lista de socios.

- **RF-02: Consulta y Edición de Socios**

- **Descripción:** El sistema debe permitir la visualización y modificación de los datos de un socio existente identificado por su identificador primario (`pk`).
- **Proceso:** Recuperar la entidad mediante consulta ORM. Permitir la actualización de campos a través de una petición HTTP POST hacia `/socios/editar/<int:pk>/`.
- **Salida:** Persistencia de cambios y redirección automática con mensaje flash.

- **RF-03: Consulta Asíncrona de Códigos Postales (SEPOMEX API Local)**

- **Descripción:** El sistema debe proveer un endpoint REST asíncrono para obtener los datos geográficos a partir de un Código Postal introducido en la interfaz.
- **Entrada:** Petición HTTP GET al endpoint `/socios/buscar-cp/?cp=<codigo_postal>`.
- **Proceso:**
 1. Validar que el parámetro `cp` no esté vacío y tenga una longitud exacta de 5 caracteres numéricos. Si es inválido, retornar HTTP 400.
 2. Consultar la tabla `catalogo_sepomex` filtrando por el índice `codigo_postal`.
 3. Si no existen registros, retornar HTTP 404 ("Código postal no encontrado").
 4. Si se encuentran coincidencias, extraer el primer registro para definir la Entidad Federativa y la Delegación/Municipio.

5. Agrupar de forma única y ordenada alfabéticamente las descripciones de las colonias asociadas.

- **Salida:** Estructura de respuesta en formato JSON con la siguiente nomenclatura técnica:

```
{  
  "estado": "Jalisco",  
  "municipio": "Guadalajara",  
  "colonias": ["Americana", "Centro", "Ladrón de Guevara"]  
}
```

3.1.2 Módulo 2: Registro e Integridad de Quejas y Reclamaciones

● RF-04: Captura de Queja y Autocompletado del Socio

- **Descripción:** El capturista debe poder asociar una queja a un socio financiero del catálogo y verificar sus datos geográficos de manera automática en tiempo real.
- **Proceso:** Al seleccionar un socio del elemento desplegable `<select id="select-institucion">`, el script Vanilla JS del frontend debe leer los atributos de metadatos del DOM (`data-estado`, `data-municipio`, `data-localidad`) e inyectarlos de inmediato en los campos de entrada correspondientes del formulario de queja.
- **Salida:** Campos de ubicación autocompletados en la vista, manteniendo el atributo de edición habilitado para posibles correcciones dinámicas antes del submit.

● RF-05: Validación Cruzada de Fecha de Cierre

- **Descripción:** El sistema debe exigir obligatoriamente una fecha de cierre cuando el estado de la queja se defina como concluido.
- **Proceso (Cliente):** Si el valor del elemento HTML `#id_estado` es igual a '2' (Concluido/Cerrado), se debe cambiar el atributo CSS de visualización del contenedor `#fecha-cierre-container` a `display: block` y definir el campo `#id_fecha_cierre` con el atributo `required = true`.
- **Proceso (Servidor):** En el método `clean()` del modelo `Queja`, se debe evaluar si el atributo numérico `estado` es igual a 2. Si es verdadero y `fecha_cierre` es nulo o vacío, el sistema debe arrojar una excepción `ValidationError({'fecha_cierre': 'La fecha de cierre es obligatoria cuando el estado es Concluido/Cerrado.'})` bloqueando la persistencia.
- **Salida:** Registro rechazado con advertencia visual en pantalla en caso de incumplimiento de la regla.

● RF-06: Generación Automática de Folio Consecutivo Regulator

- **Descripción:** El sistema debe asignar automáticamente a cada reclamación guardada por primera vez un identificador unívoco y no editable basado en la fecha de consulta.
- **Proceso:** Al ejecutarse el método `save()` del modelo `Queja`, si el atributo `folio` no tiene valor asignado:
 1. Extraer los últimos dos dígitos del año (YY) e inyectar el mes con relleno a dos caracteres (MM) de la fecha del atributo `fecha_consulta`.

2. Realizar un conteo indexado sobre la base de datos de las quejas registradas en el mismo año y mes.
 3. Incrementar el contador en una unidad (`consecutivo = count + 1`).
 4. Concatenar los valores estructurados en formato `YYMMNN` (donde `NN` se formatea a dos dígitos fijos, por ejemplo, `260601` para el primer registro de junio de 2026).
- **Salida:** Asignación automática del folio al objeto `Queja` persistido en la base de datos relacional.

3.1.3 Módulo 3: Generación de Acuses y Cierre Trimestral

● RF-07: Vista Imprimible de Voucher / Ticket de Queja

- **Descripción:** El sistema debe ofrecer una pantalla de confirmación optimizada para su impresión física o exportación digital que sirva como comprobante para el cliente.
- **Proceso:** Al acceder a la ruta `/captura/ticket/<str:folio>/`, el sistema procesa una consulta en base de datos para obtener la información de la queja y el socio. El archivo CSS asociado emplea una media query del tipo `@media print` para:
 1. Ocultar la barra de navegación `.ucisa-navbar`.
 2. Ocultar botones de acción física del navegador.
 3. Ocultar contenedores de depuración.
 4. Estilizar los bordes de la tarjeta del voucher oficial para ajustarse al estándar de impresión de tickets.
- **Salida:** Interfaz HTML limpia lista para diálogo nativo de impresión (`window.print()`).

● RF-08: Filtrado Temporal para Lotes Trimestrales

- **Descripción:** El sistema debe permitir filtrar el histórico de reclamaciones según el año y el trimestre del año fiscal para facilitar la auditoría previa al reporte de CONDUSEF.
- **Proceso:** A través de una petición GET al controlador `cierre_trimestral`, filtrar las quejas según los meses correspondientes al trimestre seleccionado:
 - Trimestre 1: Enero (1), Febrero (2), Marzo (3).
 - Trimestre 2: Abril (4), Mayo (5), Junio (6).
 - Trimestre 3: Julio (7), Agosto (8), Septiembre (9).
 - Trimestre 4: Octubre (10), Noviembre (11), Diciembre (12).
- **Salida:** Lista de reclamaciones representadas en una tabla HTML dinámica con checkboxes de selección individual y colectiva.

● RF-09: Exportación e Integridad de Lote JSON

- **Descripción:** El sistema debe empaquetar de forma atómica los datos de las reclamaciones seleccionadas en un archivo descargable con extensión `.json`.
- **Proceso:** Al recibir una petición POST con un arreglo de identificadores `queja_ids`:

1. Consultar en base de datos las reclamaciones asociadas a dichos folios.
 2. Construir una estructura de datos lineal (lista de diccionarios Python) mapeando los atributos: `folio`, `fecha_consulta` (formato ISO YYYY-MM-DD), `estado` (código numérico), `fecha_cierre` (formato ISO o null), `producto` (clave numérica oficial de Producto), `causa` (clave numérica oficial de Causa), y `medio` de recepción.
 3. Serializar la estructura de datos en formato JSON formateado con indentación de 4 espacios.
- **Salida:** Retorno de una respuesta `JsonResponse` con cabecera de cabecera HTTP `Content-Disposition: attachment; filename="lote_condusef.json"` que fuerza la descarga directa del archivo en el sistema operativo del cliente.
-

3.2 Requisitos No Funcionales

3.2.1 Rendimiento y Eficiencia

- **RNF-01 (Tiempos de Respuesta de Consultas Geográficas):** La consulta asíncrona por código postal (`buscar_cp`) debe ejecutarse y retornar su respuesta JSON en un tiempo inferior a 150 milisegundos bajo condiciones de carga normales (red local). Esto se garantiza mediante la creación de un índice físico explícito denominado `idx_codigo_postal` sobre la columna `codigo_postal` en la tabla `catalogo_sepomex` de SQLite.
- **RNF-02 (Carga Masiva Eficiente):** El proceso de carga inicial e importación del catálogo SEPOMEX desde fuentes Excel de gran volumen (más de 10,000 filas) debe utilizar transacciones atómicas agrupadas (`django.db.transaction.atomic()`) y la API `bulk_create` de Django ORM con lotes fijos (*batch size* de 5000 registros). Esto evita la degradación de memoria del servidor y previene bloqueos por exceso de escrituras individuales en el motor de base de datos relacional.

3.2.2 Seguridad e Integridad de Datos

- **RNF-03 (Protección contra Ataques CSRF):** Todas las operaciones de modificación de estado (POST) en el sistema, incluyendo el alta de socios y la generación de lotes JSON, deben estar firmadas por un token criptográfico único inyectado dinámicamente mediante el decorador y tag `{% csrf_token %}` en la plantilla del lado del servidor de Django. Las peticiones que no cuenten con un token válido serán abortadas con código de estado HTTP 403 Forbidden.
- **RNF-04 (Protección contra Inyección SQL):** El sistema debe abstraer completamente las consultas de escritura y lectura mediante el gestor de consultas relacional (QuerySets) de Django ORM. Queda estrictamente prohibido el uso de consultas SQL crudas en

formato de strings concatenados, garantizando la sanitización automática de variables ante ataques de inyección SQL.

3.2.3 Confiabilidad y Disponibilidad

- **RNF-05 (Integridad Transaccional):** En caso de fallas a nivel de hardware, red o de sistema operativo durante la generación del lote trimestral o la inserción masiva de registros geográficos, el motor relacional SQLite debe garantizar el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) a través del archivo de diario de transacciones persistido en disco.

3.2.4 Mantenibilidad y Escalabilidad

- **RNF-06 (Modularidad de la Arquitectura):** El código fuente del proyecto debe estar estructurado bajo el paradigma de separación de responsabilidades y modularidad de Django. La lógica del negocio debe estar segmentada en aplicaciones internas aisladas (módulo de `apps.consultas` y módulo de `apps.socios`), permitiendo que futuras integraciones (por ejemplo, pasarelas de comunicación API directas con CONDUSEF) puedan ser implementadas como módulos adicionales sin afectar el funcionamiento del núcleo del aplicativo.
- **RNF-07 (Portabilidad mediante Contenedores):** El sistema debe ser configurable para ser empaquetado, distribuido y ejecutado en cualquier sistema operativo cliente o servidor que cuente con Docker y Docker Compose, a través de archivos de configuración estandarizados (`Dockerfile` con imagen base ligera de Python 3.12 y `docker-compose.yml` para orquestación local).